

Parsers — Design, Responsibilities and Implementation

1. Purpose and Scope

Parsers are deterministic, grammar-driven components responsible for converting a normalized transaction narrative into a structured set of factual fields. They only extract what is explicitly present in the narrative.

Parsers do not infer meaning, assign payer or payee roles, resolve aliases, or apply business logic. Those responsibilities belong to downstream components such as `extract_payer_payee()` and the Alias Engine.

In Project Gamma, parsers are always invoked through the CTPTY pipeline:

None

Narrative → Format Detection → Parser → Payer/Payee Extraction → Aliasing

Parsers are not intended to be called directly by external systems.

2. Design Goals and Principles

Parsers in this system are designed with the following principles:

- **Deterministic behavior** — the same input always produces the same output.
- Conservative extraction — if a value is ambiguous, it is omitted.
- Strict separation of concerns — parsers extract facts only.

- **Format isolation** — each parser handles one well-defined family of narratives.
- Human-in-the-loop friendliness — unknown or ambiguous patterns are surfaced, not silently learned.
- Production safety — grammar rules are explicit and delimiter-aware to minimize false positives.

3. Parser Contract

Every parser follows a common conceptual interface:

- **is_<format>(line: str)**
 - Determines whether the narrative matches the parser's format.
 - Some formats (such as LATAM) return a pattern identifier instead of a boolean.
- **parse_<format>(line: str)**
 - Extracts fields and returns a dictionary, or {} / None if parsing is not reliable.

Parser outputs:

- Always preserve the original raw narrative.
- Include only explicit fields found in the text.
- Never assign payer or payee roles.

4. Routing and Precedence

Format detection and routing are centralized. When multiple formats could match, precedence rules ensure the most specific parser is chosen.

The table below shows the typical precedence order for routing different parser types:

Routing is intentionally conservative. If no parser matches confidently, the system prefers to return no structured data rather than incorrect data.

5. Key-Based Grammar Pipeline (V1 / V2)

Applies only to:

- WIRE
- SWIFT
- ACH
- ALL (generic)

These four formats consistently use delimiter-based labeled fields, making them suitable for a shared grammar pipeline. Other formats do not use this pipeline.

Why a two-stage pipeline exists

Many bank narratives contain both top-level keys and embedded inline keys inside values. The V1/V2 approach handles this cleanly and predictably.

V1 — Standalone key extraction

- Uses a canonical **KEYS** list
- Keys must be delimiter-bounded (=, :, /, whitespace, etc.)
- Extracts top-level segments into key → raw_value

V2 — Inline key expansion

- Applies **INLINE_KEYS** inside extracted values
- Splits embedded structures such as account numbers, purpose codes, and identifiers
- Produces nested dictionaries when applicable

Result

The pipeline returns a structured, delimiter-faithful representation of the narrative without guessing intent. This pipeline is not reused for pattern-based or free-text formats.

★ Some additional details about each type of parser →

6. All Parser (Generic)

The **ALL** parser is the foundational grammar used by WIRE, ACH, and SWIFT-like messages and **also acts as a robust fallback**.

It extracts:

- Every recognized standalone key
- Inline sub-keys inside values

It is structural by design and does not guarantee semantic correctness. Its primary role is to provide maximum factual visibility.

7. WIRE Parser

The **WIRE** parser handles domestic and international wire transfer narratives.

Typical extractions include:

- Bank identifiers (ABA, IBK, BIC)
- Account numbers
- Originator and beneficiary fragments
- Reference IDs (TRN, UETR, PARTIALREF)
- Dates and timestamps

Names are frequently truncated and should not be corrected or inferred at the parser level. Identity resolution is deferred to the Alias Engine.

8. SWIFT Parser

The **SWIFT** parser handles SWIFT-style tag-based messages, even when flattened into a single line. It maps tags to canonical fields and preserves raw tag content for auditability. The parser does not attempt inference when tags are missing or malformed.

9. ACH Parser

The **ACH** parser extracts structured information from ACH narratives, which are typically batch-oriented. Common fields include:

- Company and originator identifiers
- Entry class and batch metadata
- Trace numbers
- Addenda text

Company names in ACH data do not necessarily indicate beneficiary or payer roles. Direction and role assignment are handled downstream.

10. LATAM Parser (Pattern-Based)

- LATAM formats are highly heterogeneous and unsuitable for a single grammar.
- Instead, parsing is routed through multiple explicit patterns (pattern1 - 12).
- Each pattern has its own recognizer and parser.

This approach allows precise, country- or product-specific handling while keeping failures observable and contained. New LATAM patterns should be added only after repeated real-world failures confirm a stable structure.

11. Processor EFTs Parser

This parser handles settlement lines from external payment processors. Characteristics:

- Processor name followed by a processor code
- Batch IDs, references, sometimes dates
- Typically reference-heavy and name-light

Strict exclusion rules prevent collisions with internal sweeps, funds transfers, and interest postings.

12. Funds Transfer Parser

The **Funds Transfer** parser handles internal bank movements such as sweeps and FRMDEP transfers. It extracts:

- From / to account numbers
- Transfer direction
- Reference numbers
- Descriptions such as fees

Correct routing is critical to avoid misclassifying processor or interest transactions.

13. Specialized and Fallback Parsers

Parsers such as **DIRECT_DEBIT**, **PAYPAL**, **VENDORPAY**, **DISBURSEMENT**, **REMITTANCE**, **MERCHREF**, and **MISC** handle narrower or less structured rails.

These parsers:

- Are intentionally conservative
- Extract only explicit facts
- Return minimal output when uncertain
- Are added only when sufficient real-world examples justify their existence

14. Failure Handling and Rule Evolution

Parsing failures are expected and are part of the system's learning cycle. When parsing fails:

1. Identify the detected format.
2. Collect failing narratives.
3. Analyze whether failure is due to:
 - Missing grammar rules
 - Delimiter changes
 - New structural variants

4. Update the parser rules or add a new parser or pattern.
5. Validate changes with unit and integration tests.

This controlled loop ensures correctness without regression.

15. Adding a New Parser

- To add a new parser:
- Collect a representative corpus of real narratives (10–200 examples) that the new parser must handle.
- Decide if the new format suits:
 - V1/V2 key pipeline (wire/ach/ swift / generic)
 - pattern-based routing (like LATAM)
 - custom parsing family (processor/funds/etc.)
- Implement **is_fmt** and **parse_fmt** in a new module under **experiments/parsers/<format>**.
- Add format detection in **util.py/detect_format()**
- Add routing logic in **route.py/route_to_parser()**
- Run integration tests via the **CTPTY()** entry point.
- Add documentation (brief description + examples) to docs/parsers.md.